Database Relationship Analysis Using Joins

Aim

To implement SQL joins and generate business reports by combining customer, product, order, and payment information from different database tables.

1. Implement INNER JOIN

Explanation:

INNER JOIN retrieves only the records that have matching values in both tables. It is used to combine related information from multiple tables.

Task 1: Display customer details along with their orders.

```
SQL> SELECT c.customer_id, c.first_name||' '||c.last_name as customer_name, c.email, o.order_id, o.order_date, o.total_amount, o.order_status FRO
ustomer c JOIN orders o ON c.customer_id = o.customer_id;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
EMAIL                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
------------------------------ ---------- --------- ------------
ORDER_STATUS
--------------------
C-2001               Lira James
lira2001@edss.com              2506       14-SEP-26        45000
Delivered

C-2002               Kenny Robert
kenny2002@edss.com             2507       10-AUG-26        75000
Shipped

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
EMAIL                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
------------------------------ ---------- --------- ------------
ORDER_STATUS
--------------------

C-2003               Kairo William
kairo2003@edss.com             2508       22-AUG-26       120000
Pending

C-2004               Suxi Martin
suxi2004@edss.com              2509       05-SEP-26        55000

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
```

Task 2: Display order details with payment information.

```
SQL> SELECT o.order_id, o.order_date, o.total_amount, o.order_status, p.payment_id, p.payment_mode, p.payment_status, p.transaction_amount FROM orde
rs o JOIN payment p ON o.order_id = p.order_id;

ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- --------- ------------ -------------------- -------
PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
-------------------- ---------- ------------------
2507       10-AUG-26        75000 Shipped              PI07
UPI                  Success                  4200

2508       22-AUG-26       120000 Pending              PI08
Cash                 Pending                  1850

2509       05-SEP-26        55000 Delivered            PI09
Card                 Failed                   6750


ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- --------- ------------ -------------------- -------
PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
-------------------- ---------- ------------------
2510       18-SEP-26        95000 Shipped              PI10
Net Banking          Success                  3200

2501       02-AUG-26         9000 Shipped              PI01
UPI                  Success                  2500

2502       12-SEP-26        80000 Delivered            PI02
Cash                 Success                  1405


ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- --------- ------------ -------------------- -------
PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
```

2. Implement LEFT JOIN

Explanation:

LEFT JOIN returns all records from the left table and matching records from the right table. If no matching record exists, NULL values are displayed for the right table.

Task 1: Display all customers, including customers who have not placed orders.

```
SQL> SELECT c.first_name as customer_name, o.order_id, p.product_name, od.quantity, od.price FROM customer c JOIN orders o ON c.customer_id = o.cust
omer_id JOIN order_details od ON o.order_id = od.order_id JOIN product p ON od.product_id = p.product_id;

CUSTOMER_NAME        ORDER_ID
-------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Lira                 2506
Exide Car Battery
        20       1800

Lira                 2501
Brake Pad Set
        15      28500

CUSTOMER_NAME        ORDER_ID
-------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------

Kenny                2507
Leather Seat Cover
         5       6500

Kenny                2502
Castrol Engine Oil
```

Task 2: Find customers without any purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME
  2  FROM CUSTOMER C
  3  LEFT JOIN ORDERS O
  4  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  5  WHERE O.ORDER_ID IS NULL;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- --------------------
C-2006               Nova
C-2010               Ash
C-2007               Kaito
C-2008               Valtor
C-2009               Infernape
```

Task 3: Display all products, including products with no sales.

```
SQL> SELECT c.customer_id, c.first_name as customer_name, o.order_id, o.order_date, o.total_amount FROM customer c LEFT JOIN orders o
 ON c.customer_id = o.customer_id;

CUSTOMER_ID          CUSTOMER_NAME        ORDER_ID   ORDER_DAT TOTAL_AMOUNT
-------------------- -------------------- ---------- --------- ------------
C-2001               Lira                 2506       14-SEP-26        45000
C-2002               Kenny                2507       10-AUG-26        75000
C-2003               Kairo                2508       22-AUG-26       120000
C-2004               Suxi                 2509       05-SEP-26        55000
C-2005               Blissy               2510       18-SEP-26        95000
C-2001               Lira                 2501       02-AUG-26         9000
C-2002               Kenny                2502       12-SEP-26        80000
C-2003               Kairo                2503       06-SEP-26       100000
C-2004               Suxi                 2504       22-SEP-26       300000
C-2005               Blissy               2505       22-OCT-26       250000
C-2006               Nova

CUSTOMER_ID          CUSTOMER_NAME        ORDER_ID   ORDER_DAT TOTAL_AMOUNT
-------------------- -------------------- ---------- --------- ------------
C-2010               Ash
C-2007               Kaito
C-2008               Valtor
C-2009               Infernape

15 rows selected.
```

3. Implement RIGHT JOIN

Explanation:

RIGHT JOIN returns all records from the right table and matching records from the left table. If a matching record is unavailable, NULL values are displayed.

Task 1: Display all orders with customer information.

```
SQL> SELECT O.*, C.* FROM CUSTOMER C RIGHT JOIN ORDERS O ON O.CUSTOMER_ID = C.CUSTOMER_ID;

ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ---------- --------- ------------ --------------------
CUSTOMER_ID          FIRST_NAME           LAST_NAME
-------------------- -------------------- --------------------
EMAIL                                PHONE CREATED_A
-------------------------------- ---------- ---------
2506       C-2001     14-SEP-26        45000 Delivered
C-2001               Lira                 James
lira2001@edss.com               9123456780 15-MAR-24

2501       C-2001     02-AUG-26         9000 Shipped
C-2001               Lira                 James
lira2001@edss.com               9123456780 15-MAR-24

ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ---------- --------- ------------ --------------------
CUSTOMER_ID          FIRST_NAME           LAST_NAME
-------------------- -------------------- --------------------
EMAIL                                PHONE CREATED_A
-------------------------------- ---------- ---------

2507       C-2002     10-AUG-26        75000 Shipped
C-2002               Kenny                Robert
kenny2002@edss.com              9234567891 21-JUN-24

2502       C-2002     12-SEP-26        80000 Delivered
C-2002               Kenny                Robert

ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
```

Task 2: Find orders where customer details are missing.

```
SQL> SELECT O.ORDER_ID, O.ORDER_DATE,
  2         C.CUSTOMER_ID, C.CUSTOMER_NAME
  3  FROM CUSTOMER C
  4  RIGHT JOIN ORDERS O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  WHERE C.CUSTOMER_ID IS NULL;

no rows selected
```

Task 3: Display all payment records with order details.

```
SQL> SELECT P.PAYMENT_ID, P.PAYMENT_MODE,
  2         P.PAYMENT_STATUS, O.ORDER_ID,
  3         O.ORDER_DATE
  4  FROM ORDERS O
  5  RIGHT JOIN PAYMENT P
  6  ON O.ORDER_ID = P.ORDER_ID;

PAYMENT PAYMENT_MODE         PAYMENT_ST ORDER_ID   ORDER_DAT
------- -------------------- ---------- ---------- ---------
PI06    Cash                 Failed     2506       14-SEP-26
PI07    UPI                  Success    2507       10-AUG-26
PI08    Cash                 Pending    2508       22-AUG-26
PI09    Card                 Failed     2509       05-SEP-26
PI10    Net Banking          Success    2510       18-SEP-26
PI01    UPI                  Success    2501       02-AUG-26
PI02    Cash                 Success    2502       12-SEP-26
PI03    Card                 Failed     2503       06-SEP-26
PI04    UPI                  Pending    2504       22-SEP-26
PI05    Net Banking          Success    2505       22-OCT-26

10 rows selected.
```

4. Retrieve Complete Order Details

Explanation:

A complete order report combines customer names, product names, quantities, order dates, total amounts, and payment status. This report provides a complete view of each order.

```
SQL> SELECT C.CUSTOMER_NAME, P.PRODUCT_NAME, OD.QUANTITY, O.ORDER_DATE, (OD.QUANTITY * OD.PRICE) AS TOTAL_AMOUNT, PY.PAYMENT_STATUS FROM C
JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODU
 PAYMENT PY ON O.ORDER_ID = PY.ORDER_ID;

CUSTOMER_NAME
--------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_ST
---------- --------- ------------ ----------
Kenny
Leather Seat Cover
         5 10-AUG-26        32500 Success

Kairo
Premium Floor Mat
        15 22-AUG-26        18000 Pending

CUSTOMER_NAME
--------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_ST
---------- --------- ------------ ----------

Suxi
LED Headlight
         6 05-SEP-26        45000 Failed

Blissy
New Car Perfume
```

Result: Complete order details were retrieved by combining information from the related tables.

5. Display Customer Purchase History

Task 1: Display all products purchased by a customer.

Explanation: This query retrieves the products purchased by a particular customer along with the quantity of each product.

```
SQL> SELECT C.*, P.*, OD.* FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID JOIN PRODUCT
 P ON OD.PRODUCT_ID=P.PRODUCT_ID WHERE C.CUSTOMER_ID='C-2001';

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A PRODUCT
---------- --------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY ORDER_DETA ORDER_ID   PRODUCT_ID   QUANTITY
------- ---------- -------------- ---------- ---------- ---------- ----------
     PRICE
----------
C-2001               Lira                 lira2001@edss.com
9123456780 15-MAR-24 P2006
Exide Car Battery

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A PRODUCT
---------- --------- -------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGOR      PRICE STOCK_QUANTITY ORDER_DETA ORDER_ID   PRODUCT_ID   QUANTITY
------- ---------- -------------- ---------- ---------- ---------- ----------
     PRICE
----------
Cat0016       6500              7 0106       2506       P2006             20
      1800


CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A PRODUCT
---------- --------- -------
```

Task 2: Calculate the total amount spent by each customer.

Explanation: This query calculates the total spending of each customer by adding the amounts from their purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(OD.QUANTITY*OD.PRICE) AS
 TOTAL_SPENT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_I
D JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID GROUP BY C.CUSTOMER_ID
, C.CUSTOMER_NAME;

CUSTOMER_ID          CUSTOMER_NAME        TOTAL_SPENT
-------------------- -------------------- -----------
C-2001               Lira                      463500
C-2002               Kenny                     119975
C-2003               Kairo                     354000
C-2004               Suxi                       95000
C-2005               Blissy                     64000
```

Task 3: Count the number of orders placed by each customer.

Explanation: This query counts the orders placed by each customer and helps identify their purchasing activity.

```
SQL> SELECT C.*, (SELECT COUNT(*) FROM ORDERS O WHERE O.CUSTOMER_ID = C.
CUSTOMER_ID) AS TOTAL_ORDERS FROM CUSTOMER C;

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A TOTAL_ORDERS
---------- --------- ------------
C-2001               Lira                 lira2001@edss.com
9123456780 15-MAR-24            2

C-2002               Kenny                kenny2002@edss.com
9234567891 21-JUN-24            2

C-2003               Kairo                kairo2003@edss.com
9345678902 10-SEP-24            2

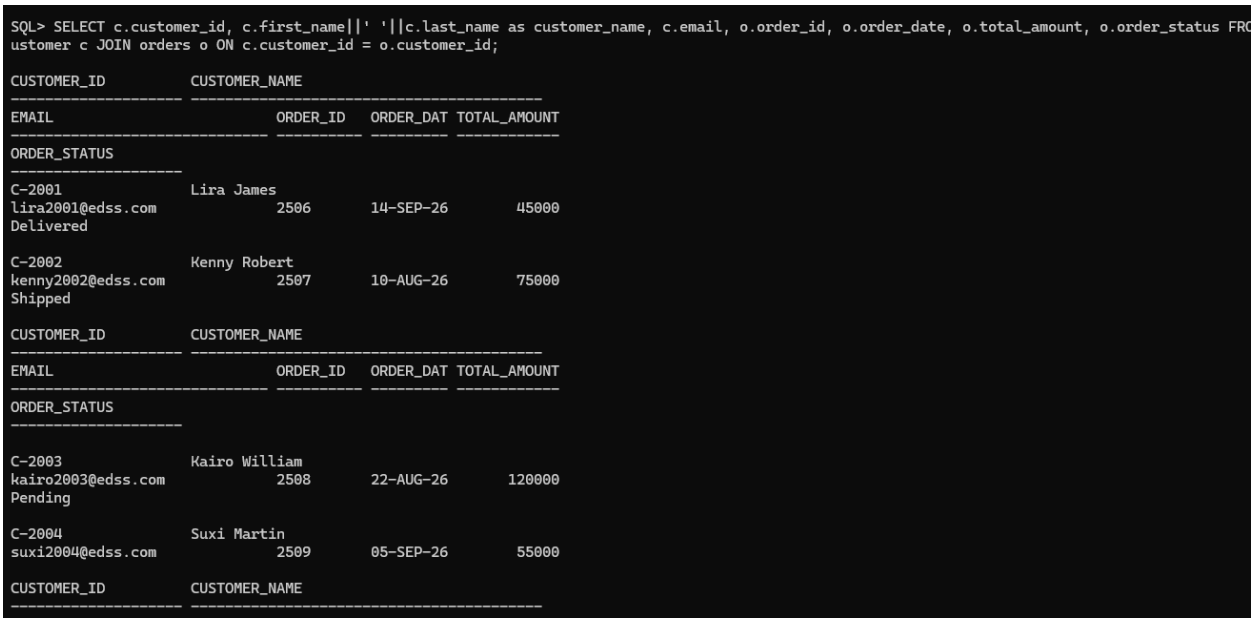

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A TOTAL_ORDERS
---------- --------- ------------
C-2004               Suxi                 suxi2004@edss.com
9456789013 18-JAN-25            2

C-2005               Blissy               blissy2005@edss.com
9786543210 27-MAY-25            2

C-2006               Nova                 nova2006@edss.com
9876543210 15-JUN-25            0


CUSTOMER_ID          CUSTOMER_NAME        EMAIL
```

Task 4: Display the latest purchase details of customers.



```
SQL> SELECT c.customer_id, c.first_name||' '||c.last_name as customer_name, c.email, o.order_id, o.order_date, o.total_amount, o.order_status FRO
ustomer c JOIN orders o ON c.customer_id = o.customer_id;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
EMAIL                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
------------------------------ ---------- --------- ------------
ORDER_STATUS
--------------------
C-2001               Lira James
lira2001@edss.com              2506       14-SEP-26        45000
Delivered

C-2002               Kenny Robert
kenny2002@edss.com             2507       10-AUG-26        75000
Shipped

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
EMAIL                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT
------------------------------ ---------- --------- ------------
ORDER_STATUS
--------------------

C-2003               Kairo William
kairo2003@edss.com             2508       22-AUG-26       120000
Pending

C-2004               Suxi Martin
suxi2004@edss.com              2509       05-SEP-26        55000

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
```

Result: Customer purchase history was analysed to identify purchased products, total spending, order counts, and latest purchases.

6. Generate Multi-Table Business Reports

Report 1: Customer Order Report

Explanation:

This report displays the customer name, order ID, order date, and order status. It helps track customer orders and their current status.

```
SQL> SELECT C.*, O.* FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.
CUSTOMER_ID;

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT
---------- --------- ---------- ---------- --------- ------------
ORDER_STATUS
--------------------
C-2001               Lira                 lira2001@edss.com
9123456780 15-MAR-24 2506       C-2001     14-SEP-26        45000
Delivered

C-2002               Kenny                kenny2002@edss.com
9234567891 21-JUN-24 2507       C-2002     10-AUG-26        75000
Shipped

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT
---------- --------- ---------- ---------- --------- ------------
ORDER_STATUS
--------------------

C-2003               Kairo                kairo2003@edss.com
9345678902 10-SEP-24 2508       C-2003     22-AUG-26       120000
Pending

C-2004               Suxi                 suxi2004@edss.com
9456789013 18-JAN-25 2509       C-2004     05-SEP-26        55000

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
```

Report 2: Sales Report

Explanation:

This report displays product names, total quantities sold, and total revenue. It helps analyse product sales and business revenue.

```
SQL> SELECT C.*, O.* FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_
ID;

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT
---------- --------- ---------- ---------- --------- ------------
ORDER_STATUS
--------------------
C-2001               Lira                 lira2001@edss.com
9123456780 15-MAR-24 2506       C-2001     14-SEP-26        45000
Delivered

C-2002               Kenny                kenny2002@edss.com
9234567891 21-JUN-24 2507       C-2002     10-AUG-26        75000
Shipped

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT
---------- --------- ---------- ---------- --------- ------------
ORDER_STATUS
--------------------

C-2003               Kairo                kairo2003@edss.com
9345678902 10-SEP-24 2508       C-2003     22-AUG-26       120000
Pending

C-2004               Suxi                 suxi2004@edss.com
9456789013 18-JAN-25 2509       C-2004     05-SEP-26        55000

CUSTOMER_ID          CUSTOMER_NAME        EMAIL
-------------------- -------------------- ------------------------------
     PHONE CREATED_A ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT
---------- --------- ---------- ---------- --------- ------------
ORDER_STATUS
--------------------
Delivered

C-2005               Blissy               blissy2005@edss.com
9786543210 27-MAY-25 2510       C-2005     18-SEP-26        95000
Shipped
```

Report 3: Payment Analysis Report

Explanation:

This report displays payment modes, the number of transactions, and successful payments. It helps analyse payment activity.

```
SQL> SELECT O.*, PY.* FROM ORDERS O JOIN PAYMENT PY ON O.ORDER_ID = PY.ORDER_ID;

ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- ---------- --------- ------------ -------------------- -------
ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
------- --------- -------------------- ---------- ------------------

2507       C-2002     10-AUG-26        75000 Shipped              PI07
2507    18-SEP-26 UPI                  Success                  4200

2508       C-2003     22-AUG-26       120000 Pending              PI08
2508    25-SEP-26 Cash                 Pending                  1850

2509       C-2004     05-SEP-26        55000 Delivered            PI09
2509    02-OCT-26 Card                 Failed                   6750


ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- ---------- --------- ------------ -------------------- -------
ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
------- --------- -------------------- ---------- ------------------

2510       C-2005     18-SEP-26        95000 Shipped              PI10
2510    05-OCT-26 Net Banking          Success                  3200

2501       C-2001     02-AUG-26         9000 Shipped              PI01
2501    02-AUG-25 UPI                  Success                  2500

2502       C-2002     12-SEP-26        80000 Delivered            PI02
2502    12-SEP-25 Cash                 Success                  1405


ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- ---------- --------- ------------ -------------------- -------
ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
------- --------- -------------------- ---------- ------------------

2503       C-2003     06-SEP-26       100000 Delivered            PI03
2503    06-SEP-25 Card                 Failed                  25500

2504       C-2004     22-SEP-26       300000 Pending              PI04
2504    22-SEP-25 UPI                  Pending                  5500

2505       C-2005     22-OCT-26       250000 Shipped              PI05
2505    22-OCT-25 Net Banking          Success                  3600


ORDER_ID   CUSTOMER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PAYMENT
---------- ---------- --------- ------------ -------------------- -------
ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
------- --------- -------------------- ---------- ------------------

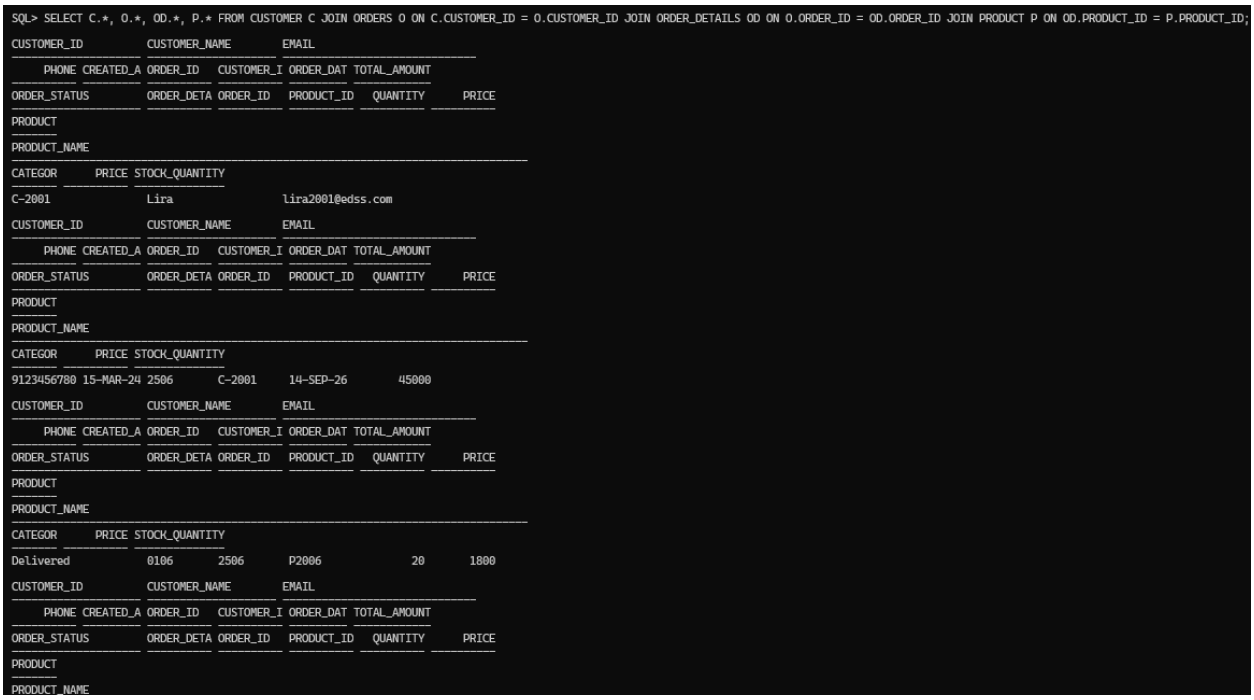
2506       C-2001     14-SEP-26        45000 Delivered            PI06
2506    14-SEP-25 Cash                 Failed                   7650


10 rows selected.
```

Report 4: Customer Purchase Analysis

Explanation:

This report identifies customers with the maximum number of orders and the highest spending. It helps understand customer purchasing behaviour.



Result: Multi-table business reports were prepared to analyse customer orders, sales, payment transactions, and customer purchasing behaviour.

Conclusion

Thus, SQL joins were studied and implemented to retrieve related information from multiple tables. Customer purchase history and business reports were used to analyse orders, sales, payments, and customer activity.